Atividade Prática — Banco de Dados Avançado

Views, Stored Procedures, Functions e Triggers em um E-commerce

Disciplina: Banco de Dados Avançado

SGBD: MySQL

Objetivo

Utilizar recursos programáveis do MySQL para implementar consultas reutilizáveis, regras de negócio, automatizações e mecanismos de integridade em um banco de dados de e-commerce.

Obrigatório: 2 Views, 2 Stored Procedures, 2 Functions e 2 Triggers.

1. Contextualização

Você foi contratado para evoluir o banco de dados de um sistema de e-commerce. O sistema já possui clientes, produtos, carrinhos, itens do carrinho, pedidos e itens dos pedidos.

O objetivo é utilizar recursos avançados do MySQL para transferir parte das responsabilidades da aplicação para o banco, garantindo reutilização de consultas, encapsulamento de regras, automatização, integridade e informações gerenciais.

2. Banco de dados fornecido

Utilize como ponto de partida o banco ecommerce.sql fornecido pelo professor.

```
customers → carts → cart_items → products
customers → orders → order_items → products
```

Principais tabelas: products (id, name, description, image, stock, price); orders (id, customer_id, total, status, paid_at, created_at, updated_at, deleted_at); order_items (id, order_id, product_id, product_name, product_price, quantity).

3. Preparação da base

Crie uma massa de testes contendo, no mínimo:

• 10 clientes.

• 15 produtos.

• 20 pedidos.

• 40 itens de pedidos.

• Diferentes situações de estoque.

• Pedidos com diferentes status.

• Clientes com nenhum, um e vários pedidos.

Utilize valores variados para preços, quantidades e datas. A massa deverá permitir testar todos os recursos.

4. Alterações na estrutura

Crie duas tabelas adicionais.

4.1. Histórico de movimentação de estoque

Crie stock_movements:

```
id | product_id | type | quantity | stock_before | stock_after | reference_type | reference_id | created_at
```

O campo type deverá permitir IN, OUT e ADJUSTMENT. reference_type poderá identificar ORDER ou MANUAL.

4.2. Histórico de alterações de pedidos

Crie order_status_history:

```
id | order_id | old_status | new_status | changed_at
```

Essa tabela será utilizada pelos Triggers.

5. Parte I — Functions

Crie pelo menos 2 Functions. Elas deverão representar cálculos ou regras reutilizáveis.

Function 1 — Total do pedido

Crie fn_order_total(order_id). Receba o ID de um pedido e retorne o total calculado a partir dos itens, considerando product_price × quantity.

```
Pedido #15
Produto A: R$ 50,00 × 2
Produto B: R$ 30,00 × 1
Total = R$ 130,00
```

Teste: execute diretamente para pelo menos três pedidos.

Function 2 — Lifetime Value do cliente

Crie fn_customer_lifetime_value(customer_id). Retorne quanto determinado cliente já gastou, considerando somente pedidos concluídos/pagos.

```
João: R$ 500,00 + R$ 250,00 + R$ 700,00 = R$ 1.450,00
```

Teste: execute para pelo menos três clientes.

6. Parte II — Views

Crie pelo menos 2 Views com informações úteis para gestão do e-commerce.

View 1 — Desempenho dos produtos

Crie vw_product_sales contendo, no mínimo: product_id, product_name, quantity_sold, revenue e average_price. A View deverá permitir identificar produtos com maior volume de vendas e considerar pedidos válidos.

View 2 — Ranking de clientes

Crie vw_customer_ranking contendo: customer_id, customer_name, orders_count, total_spent, average_order_value e last_order_at. Ordene pelo valor total gasto, do maior para o menor.

7. Parte III — Stored Procedures

Crie pelo menos 2 Stored Procedures para operações com múltiplas etapas ou regras de negócio.

Procedure 1 — Recalcular pedido

Crie sp_recalculate_order(order_id). Ela deverá verificar a existência do pedido, recalcular seu valor, atualizar orders.total e retornar informações do pedido. Utilize a Function fn_order_total().

```
CALL sp_recalculate_order(10);
```

Procedure 2 — Finalizar pedido

Crie sp_finalize_order(order_id). Ela deverá verificar existência, status e itens; recalcular o valor; alterar o status; registrar paid_at; utilizar Transaction e tratamento de erros. Em caso de falha, use ROLLBACK; em sucesso, COMMIT. Utilize a Function de cálculo.

8. Parte IV — Triggers

Crie pelo menos 2 Triggers para automatizar regras que não devem depender exclusivamente da aplicação.

Trigger 1 — Controle automático de estoque

Associe à tabela order_items. Ao inserir um item, verifique estoque, rejeite a operação se insuficiente, reduza o estoque e registre a movimentação em stock_movements.

```
Estoque: 20
Venda: 3
Novo estoque: 17
```

Quando a quantidade solicitada for superior ao estoque, gere erro usando, por exemplo, SIGNAL SQLSTATE '45000'.

Trigger 2 — Histórico de alteração de status

Associe à tabela orders. Quando o status mudar, registre automaticamente order_id, old_status, new_status e changed_at em order_status_history.

```
Pedido 25: open → paid
```

Desafio adicional — Trigger de consistência

Além dos dois Triggers obrigatórios, implemente um Trigger AFTER UPDATE ON order_items para tratar alterações de quantidade. Se aumentar, desconte apenas a diferença; se diminuir, devolva a diferença. Registre também a movimentação.

```
Quantidade: 2 → 5
Diferença descontada do estoque: 3
```

9. Integração entre os recursos

Os quatro recursos não devem ser implementados como exercícios isolados. Eles deverão trabalhar em conjunto.

```
FUNCTIONS → cálculos reutilizáveis
VIEWS → consultas reutilizáveis
PROCEDURES → operações e regras de negócio
TRIGGERS → automatização e integridade
```

A solução deverá demonstrar a combinação dos recursos.

10. Cenários de teste obrigatórios

A entrega deverá conter testes demonstrando o funcionamento dos recursos.

Teste 1 — Function de pedido

Execute para pelo menos três pedidos e demonstre os resultados.

Teste 2 — Function de cliente

Execute para pelo menos três clientes e demonstre os resultados.

Teste 3 — View de produtos

Execute SELECT * FROM vw_product_sales; e identifique o produto mais vendido e o de maior faturamento.

Teste 4 — View de clientes

Execute SELECT * FROM vw_customer_ranking; e identifique o cliente que mais gastou, o de maior número de pedidos e o ticket médio.

Teste 5 — Procedure de recálculo

Altere manualmente um item e execute CALL sp_recalculate_order(...);. Demonstre a atualização do total.

Teste 6 — Procedure de finalização

Execute CALL sp_finalize_order(...);. Demonstre status, paid_at, total e histórico.

Teste 7 — Trigger de estoque

Registre estoque antes, adicione item e demonstre estoque depois e movimentação registrada.

Teste 8 — Erro de estoque

Tente inserir quantidade superior ao estoque. Demonstre que a operação foi rejeitada e o erro retornado.

Teste 9 — Trigger de alteração de status

Altere o status e demonstre o registro automático em order_status_history.

11. Requisitos mínimos

A entrega deverá obrigatoriamente conter:

Recurso	Quantidade mínima
Views	2
Stored Procedures	2
Functions	2
Triggers	2
Tabelas novas	2
Cenários de teste	9

• pelo menos uma Procedure deverá utilizar uma Function;

• pelo menos uma Procedure deverá utilizar Transaction;

• pelo menos um Trigger deverá utilizar SIGNAL;

• pelo menos um Trigger deverá registrar informações em uma tabela de histórico;

• deverá existir controle automático de estoque;

• deverá existir histórico de alteração de status;

• todas as funcionalidades deverão ser demonstradas com dados reais da base.

12. Requisitos técnicos

Todos os objetos deverão ser criados utilizando SQL.

```
CREATE VIEW
CREATE FUNCTION
CREATE PROCEDURE
CREATE TRIGGER
```

Não será permitido substituir os recursos solicitados por código da aplicação. Se o requisito solicita uma Function, por exemplo, o cálculo deverá efetivamente existir no banco de dados.

13. Organização da entrega

A entrega deverá ser composta por um arquivo ecommerce_avancado.sql. Ele deverá conter:

• criação das tabelas adicionais;

• criação da massa de dados;

• criação das Functions;

• criação das Views;

• criação das Procedures;

• criação dos Triggers;

• comandos de teste;

• comentários explicando as principais decisões.

Organização recomendada:

```
01 - Estrutura adicional
02 - Dados
```

```
03 - Functions
04 - Views
05 - Procedures
06 - Triggers
07 - Testes
```

14. Relatório

Além do arquivo SQL, entregue um pequeno relatório contendo:

14.1. Functions

• objetivo;

• parâmetros;

• retorno;

• regra implementada;

• exemplo de utilização.

14.2. Views

• objetivo;

• tabelas utilizadas;

• informações disponibilizadas;

• exemplo de consulta.

14.3. Procedures

• objetivo;

• parâmetros;

• etapas executadas;

• tratamento de erros.

14.4. Triggers

• tabela associada;

• evento;

• momento de execução (BEFORE ou AFTER);

• regra implementada;

• impacto sobre os dados.

15. Resultado esperado

Ao final, o banco deverá ser capaz de realizar parte relevante das regras de negócio do e-commerce diretamente no SGBD.

```
VIEW → consultas reutilizáveis
FUNCTION → cálculos reutilizáveis
STORED PROCEDURE → operações e regras de negócio
TRIGGER → automatização e integridade
```

O objetivo é compreender quando, por que e com quais consequências cada recurso deve ser utilizado em um sistema real.